SHACL Validation

Simon Frost

Overview

SHACL (Shapes Constraint Language) is a W3C standard for validating RDF graphs against a set of shape constraints. RDFLib.jl includes a SHACL validator that checks whether data conforms to declared shapes.

```
using RDFLib
ex = Namespace("http://example.org/")
sh = Namespace("http://www.w3.org/ns/shacl#")
```

```
Namespace("http://www.w3.org/ns/shacl#")
```

Defining Shapes

Shapes describe the expected structure of your data — required properties, value types, cardinality, etc.

```
shapes = parse_rdf("""
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:PersonShape a sh:NodeShape ;
        sh:targetClass ex:Person ;
        sh:property [
            sh:path ex:name ;
            sh:minCount 1 ;
            sh:maxCount 1 ;
            sh:datatype xsd:string ;
            sh:name "name" ;
```

```
        ] ;
        sh:property [
            sh:path ex:age ;
            sh:minCount 1 ;
            sh:datatype xsd:integer ;
            sh:minInclusive 0 ;
            sh:maxInclusive 150 ;
            sh:name "age" ;
        ] ;
        sh:property [
            sh:path ex:email ;
            sh:maxCount 1 ;
            sh:pattern "^[^@]+@[^@]+\\\\.[^@]+\$" ;
            sh:name "email" ;
        ] .
""", TurtleFormat())

println("Shapes graph: $(length(shapes)) triples")
```

```
Shapes graph: 20 triples
```

Validating Conforming Data

```
valid_data = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:alice a ex:Person ;
        ex:name "Alice"^^xsd:string ;
        ex:age 30 ;
        ex:email "alice@example.org" .
""", TurtleFormat())

report = validate(valid_data, shapes)
println("Conforms: ", report.conforms)
println("Violations: ", length(report.results))
```

```
Conforms: true
Violations: 0
```

Detecting Violations

```
invalid_data = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    # Missing required name
    ex:bob a ex:Person ;
        ex:age 25 .

    # Age out of range
    ex:carol a ex:Person ;
        ex:name "Carol"^^xsd:string ;
        ex:age -5 .

    # Multiple names (violates maxCount)
    ex:dave a ex:Person ;
        ex:name "Dave"^^xsd:string ;
        ex:name "David"^^xsd:string ;
        ex:age 40 .
""", TurtleFormat())

report = validate(invalid_data, shapes)
println("Conforms: ", report.conforms)
println("Number of violations: ", length(report.results))

for r in report.results
    println("\nViolation:")
    println("  Focus node: ", r.focus_node)
    println("  Property:   ", r.path)
    println("  Message:    ", r.message)
end
```

```
Conforms: false
Number of violations: 3

Violation:
  Focus node: URIRef("http://example.org/dave")
  Property:   URIRef("http://example.org/name")
  Message:    Expected at most 1 values for http://example.org/name, got 2
```

```
Violation:
  Focus node: URIRef("http://example.org/carol")
  Property:   URIRef("http://example.org/age")
  Message:    Value -5 < minInclusive 0.0

Violation:
  Focus node: URIRef("http://example.org/bob")
  Property:   URIRef("http://example.org/name")
  Message:    Expected at least 1 values for http://example.org/name, got 0
```

Shape Constraint Types

Datatype Constraints

```
shapes_dt = parse_rdf("""
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:MeasurementShape a sh:NodeShape ;
        sh:targetClass ex:Measurement ;
        sh:property [
            sh:path ex:value ;
            sh:datatype xsd:decimal ;
            sh:minCount 1 ;
        ] ;
        sh:property [
            sh:path ex:unit ;
            sh:datatype xsd:string ;
            sh:minCount 1 ;
        ] .
""", TurtleFormat())

data = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:m1 a ex:Measurement ;
        ex:value "98.6"^^xsd:decimal ;
        ex:unit "fahrenheit"^^xsd:string .
```

```
    ex:m2 a ex:Measurement ;
        ex:value "not a number" ;
        ex:unit "celsius"^^xsd:string .
""", TurtleFormat())

report = validate(data, shapes_dt)
println("Conforms: ", report.conforms)
for r in report.results
    println("  $(r.focus_node): $(r.message)")
end
```

```
Conforms: false
  URIRef("http://example.org/m2"): Expected datatype http://www.w3.org/2001/XMLSchema#decimal
```

Class Constraints

```
shapes_cls = parse_rdf("""
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .

    ex:EmployeeShape a sh:NodeShape ;
        sh:targetClass ex:Employee ;
        sh:property [
            sh:path ex:worksFor ;
            sh:class ex:Organization ;
            sh:minCount 1 ;
        ] .
""", TurtleFormat())

data = parse_rdf("""
    @prefix ex: <http://example.org/> .

    ex:acme a ex:Organization .
    ex:alice a ex:Employee ;
        ex:worksFor ex:acme .
    ex:bob a ex:Employee ;
        ex:worksFor ex:unknownCorp .
""", TurtleFormat())

report = validate(data, shapes_cls)
```

```
println("Conforms: ", report.conforms)
for r in report.results
    println("  $(r.focus_node): $(r.message)")
end
```

```
Conforms: false
  URIRef("http://example.org/bob"): Value does not have required rdf:type http://example.org/
```

String Length and Pattern

```
shapes_str = parse_rdf("""
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:UsernameShape a sh:NodeShape ;
        sh:targetClass ex:User ;
        sh:property [
            sh:path ex:username ;
            sh:minLength 3 ;
            sh:maxLength 20 ;
            sh:pattern "^[a-zA-Z][a-zA-Z0-9_]*\$" ;
            sh:minCount 1 ;
        ] .
""", TurtleFormat())

data = parse_rdf("""
    @prefix ex: <http://example.org/> .

    ex:user1 a ex:User ; ex:username "alice_dev" .
    ex:user2 a ex:User ; ex:username "ab" .
    ex:user3 a ex:User ; ex:username "123invalid" .
""", TurtleFormat())

report = validate(data, shapes_str)
println("Conforms: ", report.conforms)
for r in report.results
    println("  $(r.focus_node) ($(r.path)): $(r.message)")
end
```

```
Conforms: false
  URIRef("http://example.org/user2") (URIRef("http://example.org/username")): String length 1
  URIRef("http://example.org/user3") (URIRef("http://example.org/username")): Value "123inval
zA-Z][a-zA-Z0-9_]*$"
```

Use Case: Validating a Data Pipeline

A common pattern is to validate incoming data before processing:

```
# Define the expected schema
pipeline_shapes = parse_rdf("""
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:SensorReadingShape a sh:NodeShape ;
        sh:targetClass ex:SensorReading ;
        sh:property [
            sh:path ex:timestamp ;
            sh:datatype xsd:dateTime ;
            sh:minCount 1 ;
            sh:maxCount 1 ;
        ] ;
        sh:property [
            sh:path ex:sensorId ;
            sh:minCount 1 ;
            sh:maxCount 1 ;
        ] ;
        sh:property [
            sh:path ex:value ;
            sh:datatype xsd:decimal ;
            sh:minCount 1 ;
        ] .
""", TurtleFormat())

# Simulate incoming data
incoming = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:r1 a ex:SensorReading ;
        ex:timestamp "2025-01-15T10:30:00"^^xsd:dateTime ;
```

```
        ex:sensorId ex:sensor42 ;
        ex:value "22.5"^^xsd:decimal .

    ex:r2 a ex:SensorReading ;
        ex:sensorId ex:sensor43 ;
        ex:value "18.0"^^xsd:decimal .
""", TurtleFormat())

report = validate(incoming, pipeline_shapes)
if report.conforms
    println("  All data valid - proceeding with pipeline")
else
    println("  Validation failed - $(length(report.results)) issue(s):")
    for r in report.results
        println("  $(r.focus_node): $(r.message)")
    end
end
```

```
  Validation failed - 1 issue(s):
   URIRef("http://example.org/r2"): Expected at least 1 values for http://example.org/timestar
```